

Light and Scent repellent system to deter Honey Badgers from the Penguin Colony

Group 31



Prepared by:

Lesego Ngoasheng

NGSLES001

Shaw Campbell

CMPSHA009

Prepared for:

EEE4113F

Department of Electrical Engineering

University of Cape Town

May 25, 2025

Declaration

1. I know that plagiarism is wrong. Plagiarism is to use another's work and pretend that it is one's own.
2. I have used the IEEE convention for citation and referencing. Each contribution to, and quotation in, this report from the work(s) of other people has been attributed, and has been cited and referenced.
3. This report is my own work.
4. I have not allowed, and will not allow, anyone to copy my work with the intention of passing it off as their own work or part thereof.



May 25, 2025

Shaw Campbell

Date



May 25, 2025

Lesego Ngoasheng

Date

Contents

List of Figures	v
1 Introduction	1
1.1 Background	1
1.2 Scope and Limitations	1
1.3 Proposed Solution	2
2 Problem Analysis	3
2.1 Subsystem Breakdown	3
2.1.1 Detection - NGSLES001	3
2.1.2 Repellent - PLLCAM001	3
2.1.3 Networking and User-Interface - CMPSHA009	4
2.2 Report Outline	4
3 Literature Review	5
3.1 African Penguin and Their Conservation Status	5
3.2 Existing Aversive Conditioning, Deterrent and Repellent methods	6
3.3 Leveraging IoT and Machine Learning	8
3.4 Cost Effective Solutions	11
3.5 Conclusion	11
4 Animal detection subsystem	13
4.1 Introduction	13
4.2 Requirements Analysis	13
4.2.1 Functional Requirements	13
4.2.2 Specifications	13
4.2.3 Acceptance Tests	14
4.2.4 Traceability matrix	14
4.3 Subsystem Design	15
4.3.1 Design decisions	15
4.4 Acceptance testing	20
4.5 Physical system	21
4.5.1 Breadboard design	21
4.5.2 Veroboard design	23
4.5.3 Test Results	24
4.5.4 Recommendations	24
4.6 Conclusion	24

5	Networking and User Interface	25
5.1	Introduction	25
5.2	Requirements Analysis	25
5.2.1	Functional Requirements	25
5.2.2	Non-Functional Specifications	26
5.2.3	Acceptance Test Procedures	26
5.3	Design Choices	28
5.3.1	Networking	28
5.3.2	User Interface	29
5.4	Final Design	31
5.4.1	Networking	31
5.4.2	User Interface	36
5.5	Results and Testing	37
5.6	Conclusion	40
6	System Interface	41
7	Conclusions	42
8	BOM, GA and Github Link	43
8.1	Bill of materials	43
8.2	GAS	44
8.3	Github Link	44
	Bibliography	45

List of Figures

4.1	Sensor flow diagram	16
4.2	System schematic	19
4.3	Breadboard design views	21
4.4	Detection system code snippet	22
4.5	System output	22
4.6	Test output	23
4.7	Veroboard circuit	23
5.1	Sequence Diagram for Registration with Server	31
5.2	Sequence Diagram for Pinging Server	32
5.3	Sequence Diagram for Detection Event	32
5.4	Peer-to-peer Network Mechanism	33
5.5	Server Architecture	34
5.6	Text File Log for Broken Camera Detection Event	35
5.7	Labelled Image from Detection	35
5.8	Flow Chart for General Behaviour of ESP32-cam Device Behaviour	36
5.9	User Interface Start Up	36
5.10	User Interface System Log	37
5.11	Detection Frame During Alert and Registered Devices Frame	37
5.12	System Log Output for ATP1 Showing Device Registration and Pings and ESP32-cam Serial Monitor Output Showing Peer List Updates	39
5.13	Terminal, Serial and UI Outputs Verifying Peer-to-Peer Routing (ATP3)	39
6.1	ESP32-CAM system intrface output	41

Chapter 1

Introduction

1.1 Background

As the EEE4113F design class of 2025, we were challenged with assisting the conservationists from BirdLife South Africa, by providing innovative solutions to some of the daily challenges they face in their attempts to conserve the critically endangered African penguins.

The African penguin population has seen a rapid decrease in the 21st century and has been classified as a critically endangered species, with predictions of reaching extinction in 2035. The conservationists pointed out various reasons for this, such as the migration of the penguin's primary source of food (sardines), leading to food scarcity, their vulnerabilities during the annual cycle of molting, and their susceptibility to predators.

After an emotionally triggering presentation by Christina Hagen, whose work has been focused on establishing a new African Penguin colony, including identifying potential sites, she currently faces the challenge of predator attacks affecting the already vulnerable penguins. As group 31, our project was aimed at assisting Christina and her team to deter predators from the colonies.

The challenge posed by Christina was 'How to keep predators, particularly honey badgers away from the penguins?' She highlighted the fact that many predator deterrent systems have been deployed globally, specifically to prevent livestock predation, however, predators have shown to become habituated and quickly adapt when there is no consequence to the deterrent method used.

After intensive research was conducted, the literature revealed unexpected characteristics of our opponent. Being 23 to 28 cm tall at the shoulder and 55 to 77cm long, these animals are renowned to be the most fearless animals in the world, taking on larger predators like lions and hyenas. They are known to have exceptional courage, strength, and aggression. Post learning about their agile and intelligent ways, we concluded that the best way to deter them would be to tailor our solution to their weaknesses.

The following limitations to the project were set out.

1.2 Scope and Limitations

- **Non-lethal Deterrents:** The system must employ deterrents that do not cause harm to predators.

- **Protection of Non-target Species:** Deterrents should not disturb non-target species, particularly the penguins.
- **Low Power Consumption:** The solution must be energy-efficient to ensure sustainability.
- **Minimal Data Usage:** The system should operate with low data requirements to reduce operational costs and complexity.
- **Ease of Installation and Portability:** The solution must be straightforward to install and easily relocatable as needed.
- **Weatherproof Design:** The system must be resistant to water and adverse weather conditions to ensure durability and continuous operation.

1.3 Proposed Solution

Based on the aforementioned, the proposed solution was to build an animal detection and repellent system that makes use of bright strobing lights and a desensitizing spray. Honey badgers are nocturnal, which means they are mostly active during the night and are thus sensitive to light. Using very bright strobing lights can cause disorientation and make them vulnerable. The literature also revealed that honey badgers heavily rely on smell for hunting; moreover, they are repelled by strong scents such as garlic, mint, vinegar, cinnamon, and citronella oil. The spray is thus used to desensitize the penguin colony and repel the predators with the strong scent.

Additionally, a networking and user interface system was built to transmit detection information to the conservationists at BirdLife, alerting them to any intruders that have been detected in the vicinity of the colony.

Our solution is thus beneficial in that it takes the literature on honey badgers into consideration and capitalizes on directly using the predator's weaknesses against them to repel them from the penguin colony, while being non-lethal. Furthermore, it ensures that the vulnerable species themselves are not distressed and that the end-users are alerted when any detection events occur.

Chapter 2

Problem Analysis

We attended a short design program at the Design school, where we practised design-thinking, analysing and breaking down problem statements, and developing solution prototypes. This program equipped us with the skills necessary to meet the user's requirements by shifting the perspective of viewing the problem through a technical lens, but rather, through a client-focused lens.

We were taken through a process of understanding and empathizing with the clients to develop tailor-made solutions that would aid them in achieving their objectives. We were also equipped with communication and presentation skills, necessary to demonstrate our proposed solution.

Initially, we decided to make use of machine learning for predator classification, but after reading the literature, we thought that machine learning did not fit the time frame and budget constraints of this project. Even if we got a model to work under some arbitrary conditions, it seemed unlikely to reflect real-world feasibility. We hence decided to put the classification task on the human user, and focus on delivering alerts as robustly as possible. Furthermore, we decided to make use of one microcontroller for the entire system as opposed to one for every subsystem, removing the complexity of data transfer latencies that may arise between the microcontrollers. This then allowed us to interface all systems through one microcontroller.

The proposed solution for this project was thus broken down into the following subsystems:

2.1 Subsystem Breakdown

2.1.1 Detection - NGSLES001

This subsystem is responsible for detecting unwanted intruders. It makes use of multi-layered sensors and microcontroller software to trigger the other two subsystems when detection has been registered. It is therefore used to provide spatial information surrounding the colony and acts as the eyes of the overall system.

2.1.2 Repellent - PLLCAM001

This subsystem involves projecting a volatile liquid that will create diffusion of an offensive scent to the honey badgers. The subsystem makes use of some kind of pump or solenoid and spray bottle that is activated upon a detection event. This subsection will also involve creating the housing for the entire system.

2.1.3 Networking and User-Interface - CMPSHA009

This subsystem will apply networking and user-interface principles to create a robust mechanism to deliver detection information to the end-user.

2.2 Report Outline

This report begins with a literature review that explores the key concepts and ideas previously introduced. Following that, each subsystem is examined in detail, covering both its design and the testing processes involved. Finally, the report concludes by presenting the overall system results, highlighting the main findings and outcomes.

Note: This report specifically focuses on the detection, networking, and user interface subsystems, as unfortunately, the repellent system could not be designed.

Chapter 3

Literature Review

3.1 African Penguin and Their Conservation Status

The African penguin (*Spheniscus demersus*) is a species endemic to the Benguela upwelling ecosystem along the southwestern coast of Africa, and it is one of only seven seabird species found in this region [1]. Once the most abundant seabird in the area, with an estimated population of 1.5 to 3 million individuals, the African penguin has faced a significant population decline in recent decades. Whittington et al. report that approximately 10–20% of mature adults in the African penguin population have stopped breeding [2]. First listed as Vulnerable in 1984 [3], the species was later reclassified as Endangered in 2010 [4]. In 2024, an alarming 77.8% population decline was reported, with some researchers predicting that the species could face extinction by 2035 if current trends continue [1]

Crawford et al. identify several factors contributing to the decline of the African penguin population, including historical egg collecting, guano scraping, changes in the abundance and distribution of key prey species (such as sardines *Sardinops sagax* and anchovies *Engraulis encrasicolus*), pollution, habitat loss, competition with fisheries, climate change, and predation both on land within colonies and at sea [5]. In 2024, a newly established African penguin colony at the De Hoop Nature Reserve was attacked by a honey badger, resulting in the death of 11 endangered penguins. Hagen notes that this predation event was 'unexpected,' as it had previously been assumed that the primary threats to the colony were leopards and caracals [6]. To better assess the impact of such predation, it is crucial to explore the unique behaviors and traits of honey badgers.

Bega describes honey badgers as “notoriously inventive and determined,” known for killing more than they can consume when they encounter prey [7]. JM Gil-Sánchez et al. note that available information on the honey badger’s ecology is limited [8]; however, it has been widely recognized that they capture most of their prey by digging with powerful, broad feet and long claws [6]. Additionally, an article by Mpala Live highlights that honey badgers are “fearless and fierce,” with highly sensitive noses used for communication [9]. H. Kruuk et al. observed that honey badgers use their keen sense of smell to locate digging sites, walking with their nose to the ground [10]. During their 22 expeditions across 1 to 2-week periods in the Sahara Desert, Gil-Sánchez et al. observed that honey badgers are primarily nocturnal, meaning they are mostly active during the night [8], suggesting that bright lights could potentially be used as a deterrent. Johnson [11] found that 77.1% of bee hives were absconded within a

week following a honey badger attack, but only 11.1% were absconded after employing motion activated light deterrents - elucidating a weakness of honey badgers being light. [12] States that honey badgers will avoid areas that are strongly lit and that have a strong smell of something they do not like such as predator manure (specifically lion manure) or citronella oil.

In conservation efforts, predator control plays a critical role in safeguarding endangered species. To mitigate predation risks, a fence was designed and constructed at the newly established De Hoop Nature Reserve African penguin colony in the Western Cape, in consultation with experts on the behavior and biology of mammalian predators [6]. However, this measure proved ineffective following a honey badger's killing spree, which resulted in the loss of 11 endangered penguins. Despite this setback, ongoing research into effective deterrence methods remains essential to protect penguin colonies from future predation. Conservationists are steadfast in their commitment to preserving these iconic seabirds, especially as the African penguin population continues its alarming decline, dropping from an estimated 1 million pairs a century ago to fewer than 9,900 pairs as of 2024 [6]. Consequently, protecting newly established colonies from predators and ensuring their long-term survival is of utmost importance.

This literature review examines various foraging strategies and animal deterrence methods, drawn from global practices including those used in agriculture and livestock management. The review aims to identify potential deterrent technologies and strategies that could be adapted to protect endangered penguin colonies from honey badger predation. Additionally, it evaluates the effectiveness and reliability of these methods, focusing on sustainable, humane solutions that could address this specific conservation problem.

3.2 Existing Aversive Conditioning, Deterrent and Repellent methods

Martin E. et al. review various aversive conditioning, repellent, and deterrent methods used to reduce livestock predation by large carnivores . Aversive conditioning involves eliminating an established, undesired behavior by associating it with a negative stimulus. Deterrents and repellents, on the other hand, include physical, chemical, and acoustic stimuli or devices designed to stop predators from engaging in unwanted behaviors or to drive them away from a particular area. Martin E. et al. further differentiate between repellents and deterrents, noting that repellents elicit an immediate response in emergency situations, while deterrents have a prolonged effect in discouraging predators from returning [13].

In addition to aversive conditioning, Bomford et al. explore the effectiveness of visual and acoustic repellents in reducing predator presence. They emphasize that these repellents must significantly diminish an animal's motivation to remain in a location to be effective. Two types of acoustic repellents are identified: biosonic and non-biosonic. Non-biosonic repellents utilize high-frequency and high-intensity (dB) sounds to create discomfort, while biosonic repellents mimic biologically relevant sounds, such

as distress calls or territorial signals, to elicit a psychological or physiological response in the target species [14].

Studies on specific deterrents have yielded mixed results. Propane exploders were found to deter coyotes for only six weeks [15], while Linhart et al. reported that strobe lights were effective for an average of 53 nights over 10 trials [16]. However, not all deterrents have the desired effect. Patnode et al. found that biosonic deterrents used on captive black bears not only failed to repel them but also triggered aggressive responses, causing them to charge at the device [17]. This highlights a significant limitation of these methods—at best, they provide short-term damage reduction [12].

Additionally, Bomford et al. note that while biologically meaningful sounds—such as those used in communication—may hold potential, their effectiveness in influencing space use remains largely untested, particularly for birds [14]. Another key drawback of visual and acoustic devices is the rapid habituation of predators. Prolonged or frequent exposure causes animals to become desensitized to the stimuli, diminishing their effectiveness over time [13]. The best results, therefore, occur when repellents are activated at random intervals, relocated frequently, and paired with additional deterrents or real threats.

Most studies on aversive conditioning have focused on coyotes and wolves, primarily by lacing deceased bait with an emetic chemical compound that induces severe nausea [13]. Gustavson et al. describe this approach as non-lethal to both predators and other animals, provided the dosages—typically lithium chloride—are correctly administered [18]. However, Martin E. et al. highlight ongoing controversy regarding the effectiveness of this method, as conditioned predators have been observed to continue killing prey even if they do not consume them [13]. Applying this technique to honey badgers presents significant challenges. Using an African penguin as bait would be ethically and practically unfeasible due to the species' vulnerability and the frequent human intervention required for their protection. Additionally, honey badgers are solitary animals rather than social predators, making it unlikely that one individual's learned aversion would influence the broader population [9].

Similarly, chemical repellents have shown limited long-term success in predator management. Lehner et al. identified several promising compounds, such as capsaicin, but found that none provided consistent, long-term efficacy in reducing coyote predation without harming livestock [19]. A study conducted by the Wildlife Research Center tested sheep collars infused with capsaicin, only to find that coyotes adapted their behavior by attacking different parts of the sheep's body, demonstrating their ability to circumvent deterrent measures [20]. While these methods showed some effectiveness in controlled settings, their reliability diminished when applied in real-world scenarios. Overall, aversive conditioning and chemical deterrents have proven largely ineffective when scaled to practical levels, particularly against adaptable predators [13].

Dalle-Molle et al., under the NWT, tested projectile repellents such as Antiriot 38 mm rubber bullets and Ferret soft-slugs on bears, reporting positive results in deterring them [21]. However, in the context

of honey badger predation, this method presents significant challenges. Given the much smaller size of honey badgers compared to bears, the impact of projectile repellents could cause unintended harm rather than simply deterring them. Furthermore, these deterrents require manual operation by trained personnel [13], making them impractical for large-scale implementation across multiple penguin colonies. **Traditional Implementation of Repellents and Deterrents** Various methods have been employed to protect crops from animals, including agricultural fences made of wire and plastic. Sumathi et al. argue that the use of lethal poisons and chemical repellents is discouraged, while natural repellents, such as beehive fences, pose significant challenges due to their lack of adaptability and potential risks to humans. Additionally, they highlight the growing use of electric fences for deterring animals from farmlands [22] and securing sensitive areas [23]. However, concerns remain regarding the risks of electrocution [22].

Kommasani et al. counter these concerns, asserting that a well-designed electric fence is safe for both animals and humans. They explain that electric fences operate by delivering discrete pulses with intervals that prevent prolonged exposure to shocks. Furthermore, they emphasize the practicality and cost-effectiveness of electric fences, as they only activate when an animal completes the circuit by making contact with the fence [22]. Despite these advantages, it remains crucial to assess whether an electric fence could pose a threat not only to honey badgers but also to the penguins it aims to protect.

Linhart et al. developed a deterrent system using a 12V battery-powered strobe light and siren encased in polyvinyl chloride, programmed to emit flashing lights and loud sounds every 6–7 minutes for 7–10 seconds. These devices were strategically placed among and around sheep, reducing livestock losses by 60% [19]. Similarly, farmers who installed lights over corrals or attached bells to sheep experienced fewer predator attacks compared to those who did not [22]. However, Fritts et al. reported that placing lights and sirens along highways yielded inconclusive results in deterring predators [24].

The use of guard dogs as a deterrent was also examined. While effective in some cases, overly domesticated dogs were found to attract bears rather than repel them, highlighting the importance of training dogs specifically for predator deterrence [25].

Mishra and Yadav argue that traditional deterrence methods have failed to account for the diverse behaviors of wildlife. They note that while these techniques may show some initial success, they are often ineffective against highly adaptable species, ultimately leading to a 20% increase in agricultural damage over time [26].

3.3 Leveraging IoT and Machine Learning

The emergence of IoT and AI has led to extensive research on integrating machine learning (ML) to enhance traditional methods for preventing animal-related damages. In this approach, animals are classified using ML algorithms, allowing deterrent methods to be tailored to specific species. Various AI-driven deterrent systems have demonstrated significant success across different applications, achieving

a 92% success rate in repelling dangerous animals [27], an 80% reduction in elephant infiltrations [28], an 85% success rate in repelling animals [29], an 88% success rate in animal deterrence [30], an 85% accuracy in animal classification [31], and an additional 88% success rate in repelling animals [32]. These systems often employ conventional methods such as biosonic repellents (e.g., ultrasound), suggesting that the incorporation of IoT and AI is a key factor in their improved effectiveness. Additionally, the ability to accurately detect and classify animals ensures that deterrents are deployed only when necessary, making these solutions more energy-efficient.

To understand the practical implications of machine learning, one can examine the system developed by [26]. Their repellent system achieved an average accuracy of 91.88%, with a precision of 87.37%, recall of 91.01%, and an F1 score of 89.83%. These results were accomplished using a Recursive Convolutional Neural Network (R-CNN) and a Single Shot Multibox Detector. However, their approach relied on an NVIDIA Jetson Nano—a relatively expensive piece of hardware—along with other microprocessors integrated throughout the system. In contrast, Akhil V et al. [33] implemented a simpler approach using an Arduino and a small dataset of approximately 100 animal images, achieving an inadequate classification accuracy of just 7–8%. This highlights that effective AI implementation requires both large datasets and advanced hardware. Additionally, the AI processing system in [32] consumes 5–10W of power, leading to relatively high energy consumption even before accounting for detection devices. Their system also employs XBee radio to enable IEEE 802.15.4 communication, further increasing costs, whereas more affordable mobile network alternatives could have been used. The researchers conclude that future iterations should be designed to tolerate a wider range of terrains and weather conditions. Since the system relies heavily on vision-based detection, adverse weather necessitates more sophisticated ML models, which, in turn, raises the overall cost of implementation.

Gor et al. [34] proposed a system called GATA, which integrates Wireless Sensor Networks (WSN) and Global Positioning System (GPS) technology for wildlife tracking and protection. Their approach utilizes automatic location and movement tracking through a GPS module, an accelerometer, and a WiFi shield. This surveillance system aims to prevent animals from straying beyond protected areas while also recording movement patterns of rare and endangered species for monitoring purposes. The proposed solution employs a general-purpose Arduino as an interface between the user and the GPS module, with the accelerometer determining movement and the WiFi shield facilitating network connectivity via the IEEE 802.11b and 802.11g Wi-Fi protocols. Initial tests on cows demonstrated the system’s flexibility and cost-effectiveness. However, while this solution effectively provides real-time location data, it would be impractical for tracking honey badgers, as they are predominantly solitary animals. Additionally, tracking the penguins themselves would not directly deter predators from attacking them.

Sumathi et al. explored 24-hour farmland surveillance using cameras combined with artificial intelligence for real-time animal detection and classification. Their system employs the ‘You Only Look Once’ (YOLO) algorithm, a neural network-based object recognition model known for its speed, precision, and adaptability. When an animal is detected, the YOLO algorithm classifies the intruder, and appropriate

corrective actions are taken based on the species identified. Additionally, images and geolocation data are sent to farmers and forest officials, and if the animal remains undeterred after a short period, stronger repellents are deployed as a backup measure [22].

Divya et al. further enhanced detection accuracy by utilizing multiple passive infrared (PIR) sensors to track animal movement [35]. This approach mitigates the challenge of animals blending into their surroundings, improving detection reliability. Through a series of experiments, the system demonstrated the ability to classify intruders within milliseconds, accurately identifying multiple objects in a single frame. The YOLO algorithm proved to be highly efficient, processing nearly 45 frames per second. Additionally, the use of a NodeMCU microcontroller ensured low energy consumption, while a playback module generated distinct sound disturbances tailored to the detected intruder type [22].

Adami presents an IoT-based solution for mitigating crop damage caused by wild animals, such as boars and deer, using non-lethal and eco-friendly repelling devices [27]. The proposed system integrates IoT, computer vision, and ultrasound technology to deter animals while emphasizing a robust remote monitoring application to ensure continuous operation in rural locations.

The system architecture is designed around LoRaWAN, a long-range, low-power communication protocol essential for deployments in areas with limited telecommunication infrastructure [35]. Each repelling device is equipped with solar panels, LoRa modules, and computer vision capabilities to detect and deter specific animal species using ultrasound. The monitoring application, developed with Docker containers, integrates multiple tools: Node-RED for flow-based programming, InfluxDB for time-series data storage, MongoDB for static data management, and Grafana for real-time visualization. This modular and scalable approach enables adaptability across various deployment scenarios, making it well-suited for large-scale agricultural applications.

Experimental trials conducted in the Regional Park of Migliarino-San Rossore-Massaciuccoli, Italy, demonstrated the system’s effectiveness. The monitoring application successfully tracked the operational status of eight devices, providing real-time alerts and visualizations of device health, battery levels, and environmental conditions such as brightness. Device status was classified as active, non-active, or warning based on message frequency, enabling quick identification of malfunctions or communication failures. The findings highlight the system’s reliability in challenging rural environments and its potential as a sustainable wildlife control solution for agriculture [27]. However, the IoT-based solutions proposed in these studies, while effective in certain applications, rely heavily on sophisticated hardware and software systems that come with high costs. Due to the complexity and expense associated with these solutions, they do not appear to be viable options for this project, which necessitates more affordable and practical approaches.

3.4 Cost Effective Solutions

Bhatta et al. describe a cost-effective system that repels animals such as elephants, cows, and zebras [36]. The effectiveness of the integrated machine learning component is not addressed, so it will not be considered relevant in this context. The system uses an Arduino microcontroller for control, PIR sensors for detection, a GSM module for SMS and phone calls, and an ultrasonic buzzer. Arduino is an open-source electronic platform that is easy to use, versatile, and highly customizable, making it a good choice. When an animal is detected, the Arduino activates the appropriate frequency to transmit and contacts the user via SMS or phone call.

The design process for this system involves identifying requirements, selecting sensors and devices, designing the circuit, developing infrastructure, configuring the Arduino, implementing the system algorithm, and evaluating the system. The advantages of this system include fast communication via phone calls, user-friendly operation with mobile devices, cost-effectiveness, and the use of the ubiquitous GSM mobile network. Kommasani et al. propose a similar solution, but they use a Raspberry Pi and Pi camera to capture an image of the intruder, as detected by PIR sensors. The image is then sent via GSM to the user, who manually activates the audio voice recorder and playback (APR) to divert the animal through sound [22].

Since only the honey badger needs to be detected in this context, complex machine learning may be unnecessary, provided the system has appropriate repellents for the honey badger. Additionally, the system might use a simple scale to estimate the presence of a medium-sized animal and infer that it is a honey badger. Given that only one animal is of interest in this instance, much of the adaptive functionality offered by AI can be hard-coded into the system specifically for honey badgers, while still utilizing IoT in a cost-efficient manner, as described in [37].

3.5 Conclusion

The literature review highlights the critical status of the African penguin (*Spheniscus demersus*) and the challenges faced in their conservation due to honey badger predation.

Existing aversive conditioning, deterrent and repellent methods, while showing some success in certain contexts have significant limitations in this context. Visual and acoustic deterrents have shown mixed results, often leading to habituation over time. Chemical repellents, though promising in controlled settings, face ethical and practical challenges at scale. Methods such as electric fences and dogs pose ethical risks of both predator and penguins.

The integration of IoT and machine learning presents a promising way to more effective and adaptive techniques. AI-driven solutions have shown high success rates, offering species specific deterrence. However, these systems often come with high costs and complexity, making them less suitable for large-scale implementation in resource-limited conservation settings.

Cost-effective solutions, such as those utilizing Arduino microcontrollers and PIR sensors, offer a more

practical approach for detecting and deterring honey badgers. These systems can be tailored to the specific needs of penguin colonies, leveraging simple yet effective technologies to provide real-time alerts and deploy deterrents. While these solutions may lack the sophistication of AI-driven systems, they offer a balance between effectiveness, affordability, and ease of implementation, making them more accessible for conservation efforts.

In conclusion, protecting African penguins from honey badger predation requires a multifaceted approach that combines traditional deterrent methods with innovative technologies. Future research should focus on developing cost-effective, scalable solutions that can be easily deployed across multiple colonies, ensuring the long-term survival of this iconic species. By leveraging both proven and emerging technologies, conservationists can create a robust defense against predation, helping to reverse the alarming decline of the African penguin population.

Chapter 4

Animal detection subsystem

Prepared by: NGSLES001

4.1 Introduction

This chapter focuses on the design of the animal detection subsystem, which aims to detect the presence of predators, thus providing spatial awareness of the penguin colony to the end user. To enhance detection accuracy and conserve power, the subsystem integrates three distinct sensors, thereby minimizing false triggers and ensuring efficient operation.

4.2 Requirements Analysis

4.2.1 Functional Requirements

Table 4.1 provides the functional requirements of the detection subsystem.

Requirement ID	Description
FR01	The subsystem needs to be operational and thus designed for reliability.
FR02	Needs to have switching means to save power during honey badger inactive periods.
FR03	Needs to provide long and short-range detection means.
FR04	The three sensors need to operate sequentially to provide useful detection information.
FR05	Needs to interface with the other subsystems to provide detection information.
FR06	Needs to make use of the provided power supply.
FR07	Needs to activate a repellent and capture an image of the detected animal.

Table 4.1: Functional requirements of the detection subsystem

4.2.2 Specifications

Table 4.2 provides specifications, defined from the requirements in table 4.1.

Specification ID	Description
SP01	The detection sensors need to be powered only at night.
SP02	Must detect the animal from >5m for long-range detection and <1m for short-range detection.
SP03	Multiple sensors must be triggered to render an animal detected.
SP04	Must provide detection information to the user through the networking interface.
SP05	Must be powered from 12V, 1A power source.
SP06	Must activate the repellent light and spray and capture an image of the animal.

Table 4.2: Specifications of the detection subsystem

4.2.3 Acceptance Tests

Table 4.3 provides the summary of the acceptance tests carried out in table 4.7 to verify the operation of the subsystem.

Acceptance Test ID	Description
AT01	Test whether the system makes use of a switching mechanism to operate only at night.
AT02	Test whether the system can provide long-range detection.
AT03	Test whether the system can provide short-range detection.
AT04	Test whether the sensors operate sequentially to reduce false triggers.
AT05	Interfaces with the other subsystems by providing detection information.
AT06	Tests whether the system operates using 12V.
AT07	Tests whether the repellent system is activated and an image is captured.

Table 4.3: Detection subsystem acceptance test

4.2.4 Traceability matrix

Table 4.4 shows how the requirements, specifications and testing procedures all link.

	Requirements	Specifications	Acceptance Test
1	FR02	SP01	AT01
2	FR03	SP02	AT02, AT03
3	FR04	SP03	AT04
4	FR05	SP04	AT05
5	FR06	SP05	AT06
6	FR07	SP06	AT07

Table 4.4: Requirements Traceability matrix

4.3 Subsystem Design

4.3.1 Design decisions

The animal detection subsystem was specifically designed to identify honey badgers. Research indicates that these animals are primarily nocturnal, and thus to align with their activity patterns, the system operates exclusively at night, conserving energy and addressing the low-power consumption requirements emphasized by BirdLife conservationists.

The detection system thus firstly makes use of a light sensor, which is used to keep the system idle during honey badger inactive times. This is a means to also protect the electronic components from excessive heating as a result of being powered 24/7, which also preserves the lifespan of the components.

As the main objective of the subsystem is to detect the presence of an animal, a motion detection sensor was used for this. The three main types of motion detectors are Passive Infrared (PIR), Microwave, and Dual Technology sensors. Based on the popularity of use, as seen from the animal detection projects in the literature review, the PIR sensor was selected. These sensors work by detecting changes in infrared radiation, primarily heat emitted by humans or animals, to detect movement.

Advantages:

- Reliable for detecting movement in a specific area.
- Affordable.
- Low power consumption.
- Simple to use.

Limitations:

- Can be susceptible to false alarms from temperature changes.
- Less reliable in outdoor environments (affected by wind/weather).

To reduce false triggering, another motion detection sensor, the IR Break Beam sensor, was integrated into the system. Unlike the PIR, this sensor makes use of an IR Transmitter, which sends out a beam of infrared light that is detected by the IR Receiver, and when an object blocks the beam, a signal is sent.

Advantages:

- Offers high precision as it detects exact moments of beam interruption.
- Fast response time.
- Knows where the break occurred and thus provides directional detection.
- Immune to ambient light.

Limitations:

- Emitter and receiver must be precisely aligned.

The IR break beam sensor is designed to be placed along vulnerable positions of the penguin colony, where the predator is likely to invade, such as along the perimeter of the fence, corners, or existing dug-out paths. The PIR sensor was thus used for long-range motion detection, and the IR break beam

for short-range detection. All three sensors were combined to produce the following sequential logic of the overall detection system:



Figure 4.1: Sensor flow diagram

The following components were considered for the design of the animal detection system.

Table 4.5: Decisions involving component selection

Component	Function	Name	Advantages	Disadvantages	Cost per part
PIR sensor	Detects movement by using infrared radiation.	HC-SR501	Input voltage: 4.5–20V. Adjustable sensitivity and time delay knobs. Digital output. Up to 7m detection range.	Sensing angle: 100°	R27
IR break beam	Detects motion through the blocking of an infrared light.	Adafruit IR break beam sensor	Input voltage: 3.3–5V. 50cm working range. Microcontroller compatible.	Requires external pull-up resistor for a digital output. LED angle: 10°	R107.07
		IR break beam sensor module	Input voltage: 5V. Microcontroller compatible. Combines both emitter and receiver onto one module. Provides digital output.		R18

Component	Function	Name	Advantages	Disadvantages	Cost per part
IR transmitter	Emits infrared rays	KY-005	Input voltage: 3.3–5V. Emitting wavelength: 940nm. Microcontroller compatible.	Requires external series resistor	R17
IR receiver	Receives infrared light	KY-022	Input voltage: 3.3–5V. Reception range: 18m.	Reception angle: 45°	R23.95
Light sensor	Detects and measures light and converts it into an electrical signal.	BH1750FVI	Input voltage: 3–5V. Provides digital light intensity output. In-built 16-bit AD converter. Complex calculation and calibration omitted. 120ms response time.	Fixed I2C address. No sleep mode, leading to high power consumption. Data range: 1–65535.	R40
		TSL2561	Input voltage: 3–5V. Provides IR sensitivity filtering. Configurable I2C address. Offers temperature compensation. 400ms response time.	Data range: 0.1–40000	R64
		APDS-9960	Input voltage: 3.3V. Offers touchless gesture sensing, ambient light, and RGB color measuring and proximity detection. Can enter idle mode.	Fixed I2C communication. High latency due to multi-functionality.	R80

Component	Function	Name	Advantages	Disadvantages	Cost per part
Microcontroller	Performs arithmetic and logic operations such as data processing and storage	Arduino Uno R3	Input voltage: 5V. 14 Digital I/O pins. Easy to use.	No built-in wireless connectivity	R525
		ESP32-CAM	Input voltage: 3.3–5V. Low power 32-bit CPU and supports multiple sleep modes. Built-in camera (OV2640, 2MP). Supports Wi-Fi and Bluetooth.	Requires external power management. 10 GPIO pins available	R182

For the light sensor, the BH1750FVI was selected because it meets the basic light monitoring requirements to meet SP01 of this project. It has similar advantages to the TSL2561, however at a cheaper price. Furthermore, it offers a higher data range and faster response time. The APDS-9960 offers complex functionalities that are unnecessary for this project.

For the PIR sensor, the HC-SR501 was selected as it offers a detection range of up to 7m, while offering the flexibility of adjusting the sensitivity and time delay after motion has been detected, which meets FR03 and SPO2. Despite its limited sensing angle, it is sufficient to detect motion from various angles, especially for the long-range detection it will be used for.

The IR break beam module was selected as opposed to the separate IR transmitter and receiver modules due to its affordability, while combining both the individual IR components into one package. Moreover, it provides a digital output, making it easy to interface with a microcontroller. The Adafruit break-beam sensor is more expensive and requires additional external circuitry, as well as proper alignment (LED angle of 10°), to function properly.

For the microcontroller, the ESP32-CAM was chosen because it comes with a built-in camera, which can then be used to capture an image of the animal once detected. It also supports wifi and Bluetooth, making it easier to transfer data between the various subsystems. It is relatively cheaper than the Arduino Uno R3 board, which does not come with these multi-functionalities and supports I2C communication, which the light sensor requires to interface with the microcontroller.

Looking at the supply voltage of 12V, a voltage regulator was required to step down the power supply voltage to the 5V maximum input voltage required by most of the aforementioned components. The

XL4015 buck converter was thus selected for this application, as it provides an output voltage of 1.25-32V and a maximum output current of 5A, tunable through the device's potentiometer.

The total current drawn by the sensors, as viewed from their datasheets, is: (*See Github repository for component datasheets*)

$$50\mu A_{PIRsensor} + 7mA_{LightSensor} + 20mA_{IRsensor} = 0.02715A$$

The calculation above shows that 1A is more than sufficient current for the entire sensor subsystem.

Table 4.6 provides a summary of the components used to build the detection system.

Component	Name
Light sensor	BH1750FVI
PIR Sensor	HC-SR501
IR break beam	IR break beam module
Microcontroller	ESP32-CAM

Table 4.6: Summary of selected components

KiCad Schematic

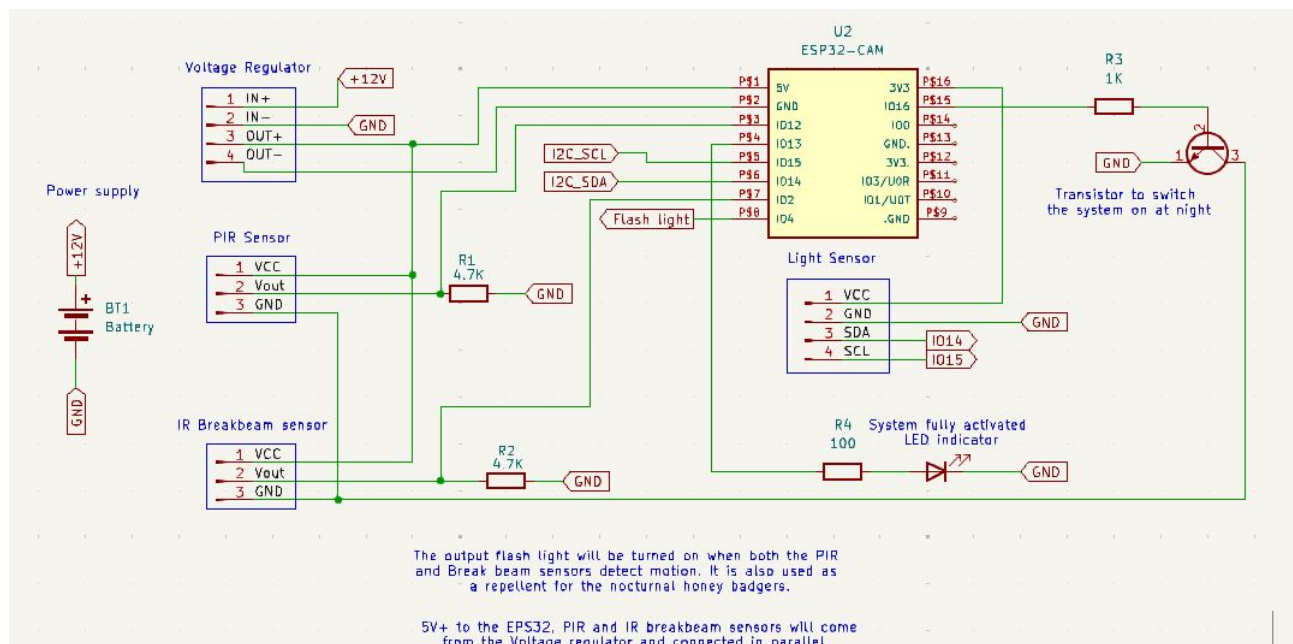


Figure 4.2: System schematic

Figure 4.2 above shows how the various sensors were connected to the ESP32-CAM microcontroller to form the overall animal detection system. A voltage regulator was used to step down the 12V power

supply to the 5V input voltage required by the sensors and ESP32-CAM. Pull-down resistors were added to the output of both the PIR and IR break beam sensors to reduce false triggering due to floating pins, which may pick up noise when the sensors are inactive. Moreover, this was also done to protect the ESP32 pins, as they are strictly 3.3V tolerant, and any high voltage from the sensors could damage the microcontroller.

Both the PIR and IR break-beam sensors are powered through the PN2222A Bipolar Junction Transistor (BJT), which acts as a low-side switch controlled by the ESP32-CAM. When the BH1750 light sensor detects low lux levels (nighttime), the ESP32 drives a small base current (I_b) into the transistor, saturating it. This allows collector-emitter current (I_c) to flow, delivering power to the sensors.

A simple LED circuit was connected to Pin 13 of the ESP32 to serve as an indicator when both the PIR and IR break beam sensors detect motion. This is thus used to indicate that an intruder has been detected.

4.4 Acceptance testing

Prior to conducting the following tests, each individual sensor must be tested using its required input voltage and a multimeter to ensure that the component is not faulty.

Once the subsystem has been built according to the schematic shown in Figure 4.2, the following tests are conducted to verify the functionality of the system.

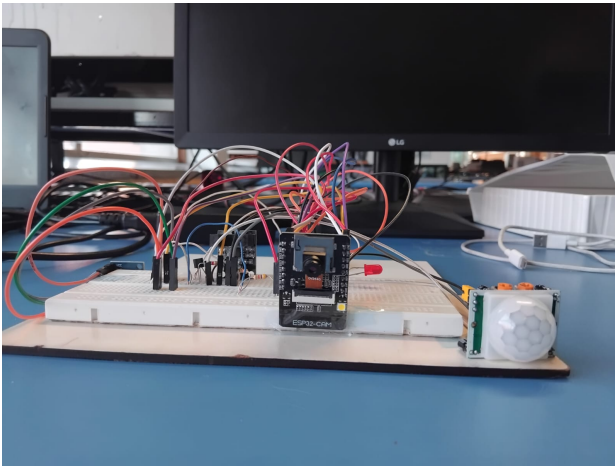
Table 4.7: Acceptance Tests

Test ID	Description	Testing Procedure	Pass/Fail Criteria
AT01	Test whether the system uses a switching mechanism to operate only at night.	Connect the sensor to its required input voltage and expose it to light. Use a multimeter to verify 0V at the transistor's base. Cover the light sensor to simulate darkness; there should be a voltage reading at the transistor's base.	When the light sensor detects light, other sensors are off. When it detects darkness, other sensors are powered on.
AT02	Test whether the system can provide long-range detection.	Connect an LED to the PIR sensor's output as an indicator. Power the PIR sensor and move at varying distances and positions relative to the sensor.	LED turns on when motion is detected.
AT03	Test whether the system can provide short-range detection.	Connect an LED to the IR break beam sensor's output as an indicator. Power the IR sensor and repeatedly place and remove an object between the emitter and receiver.	LED turns on when the beam is broken.

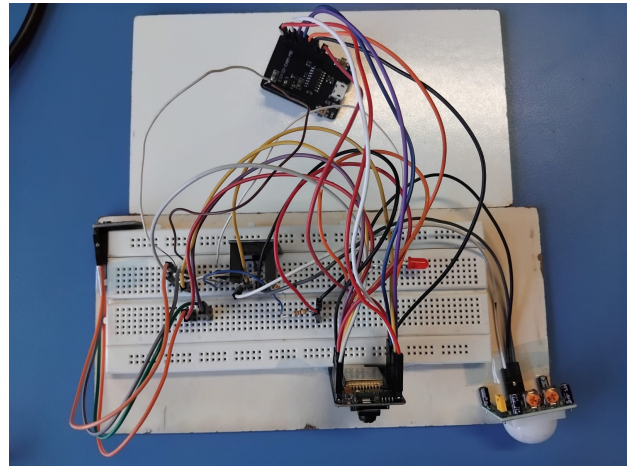
Test ID	Description	Testing Procedure	Pass/Fail Criteria
AT04	Test whether the sensors operate sequentially to reduce false triggers.	With the circuit connected as in Figure 4.2 (with 5V input), cover the light sensor as in AT01, move at a reasonable distance from the PIR sensor for motion detection as in AT02, and place an object through the IR beam to break it as in AT03.	External LED switches on.
AT05	Interface with other subsystems by providing detection information.	Use a variable in code to store when motion is detected, which can be accessed by other subsystems.	Variable successfully stores detection information accessible by other subsystems.
AT06	Test whether the system operates using 12V.	Connect a 12V power supply to the voltage regulator and adjust it using the potentiometer to supply 5V. Connect the regulator output to the circuit.	Voltage regulator provides 5V to the sensors, and the system operates as expected.
AT07	Test whether the repellent system is activated and an image is captured.	Repeat the detection sequence as in AT04. Using code, turn on the ESP32-CAM's flash and capture an image.	When motion is detected, the flash turns on, and an image is captured.

4.5 Physical system

4.5.1 Breadboard design



(a) Breadboard circuit: Side view



(b) Breadboard circuit: Top view

Figure 4.3: Breadboard design views

Figure 4.4 presents a snippet of the ESP32-CAM code used to evaluate the system's overall functionality. (See *Github repository* for full system code)

```

97 // Read PIR sensor (active HIGH when motion detected)
98 bool currentPirState = digitalRead(PIR_PIN) == HIGH;
99
100 // Detect new motion
101 if (currentPirState && !pirTriggered) {
102     pirTriggered = true;
103     lastPirTrigger = millis();
104     digitalWrite(FLASH_PIN, HIGH);
105     Serial.println("MOTION DETECTED - Flash ON");
106 }
107
108 // Read IR break beam (now HIGH when beam is broken)
109 bool beamBroken = digitalRead(IR_BREAK_PIN) == HIGH;
110
111 // Control external LED based on both sensors
112 if (pirTriggered && beamBroken) {
113     digitalWrite(EXT_LED_PIN, HIGH);
114     Serial.println("DUAL TRIGGER - External LED ON");
115 } else {
116     digitalWrite(EXT_LED_PIN, LOW);
117 }
118
119 // Reset PIR trigger after duration
120 if (pirTriggered && (millis() - lastPirTrigger > PIR_TRIGGER_DURATION)) {
121     pirTriggered = false;
122     digitalWrite(FLASH_PIN, LOW);
123     Serial.println("MOTION TIMEOUT - Flash OFF");

```

Figure 4.4: Detection system code snippet

The following output was obtained from testing the system on the breadboard.

```

SYSTEM OFF - Bright conditions
Light: 133.33 lx
Light: 133.33 lx
Light: 7.50 lx
SYSTEM ON - Dark conditions
PIR sensor warming up...
Light: 7.50 lx
MOTION DETECTED - Flash ON
Light: 8.33 lx
Light: 7.50 lx
Light: 7.50 lx
DUAL TRIGGER - External LED ON
Capturing image!

```

Figure 4.5: System output

Figure 4.5 above shows the system starting off in bright conditions, with the light sensor indicating lux levels >100 (used as the threshold for bright conditions), thus the rest of the system remains off, with the ESP32-CAM in idle mode. Once the lux levels fall beneath 20 (used as the threshold for dark conditions), the system assumes nighttime and powers on. The PIR sensor takes a few seconds

(30-60s) to warm up; during this time, it calibrates its infrared sensor to the ambient temperature and environment.

The PIR sensor then detects motion and the ESP32-CAM flash light turns on to repel the honey badgers (simulating the bright strobing repellent lights). Once the IR break beam is also broken, a dual trigger is registered, and the external LED is switched on to indicate that motion has been detected and an image is captured thereafter.

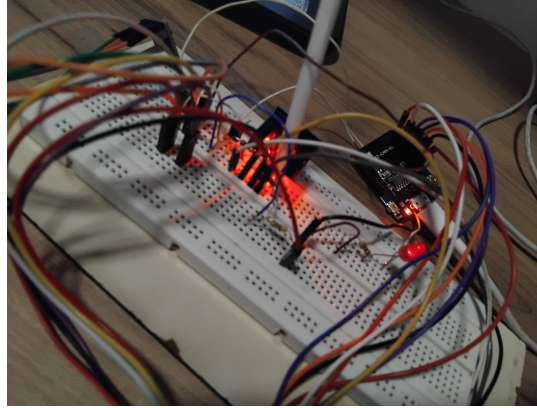


Figure 4.6: Test output

The output obtained above thus verifies the functionality of the overall detection subsystem.

4.5.2 Veroboard design

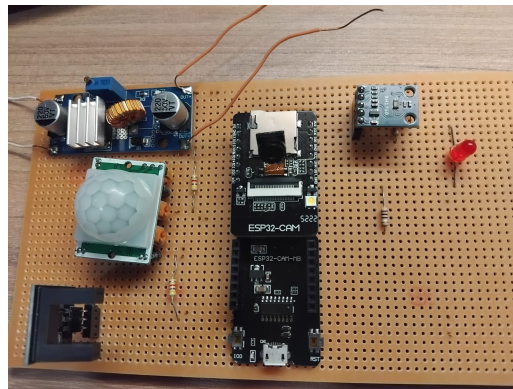


Figure 4.7: Veroboard circuit

The breadboard circuit design was transferred onto a veroboard for a more reliable and permanent solution.

4.5.3 Test Results

Test ID	Pass/Fail
AT01	Pass
AT02	Pass
AT03	Pass
AT04	Pass
AT05	Pass
AT06	Fail
AT07	Pass

Table 4.8: Acceptance Test results

While testing the subsystem on the veroboard, it was found that using the voltage regulator somewhat led to the microcontroller being damaged, despite the regulator’s output voltage being tuned to 5V, which is the input voltage of the ESP32-CAM. Moreover, while the system successfully meets its main objective of detection, it does not meet the functional requirement of using the 12V power supply, due to difficulties experienced with powering the ESP32-CAM through the voltage regulator.

With the challenge of only having two microcontrollers for this project, the risk of connecting the voltage regulator to the circuit alongside the microcontroller was again not taken, which led to us resorting to powering the system via USB cable. Despite this not being ideal, it was safer to protect the secondary microcontroller.

4.5.4 Recommendations

The IR break beam sensor module used in this project was quite small, making it challenging to demonstrate how it could be scaled for perimeter detection along a fence. To better represent real-world applications, it’s recommended to use separate transmitter and receiver units, like the Adafruit IR break beam sensors, which can be spaced appropriately based on the actual size of a honey badger.

Additionally, while the system includes a voltage regulator to step down the power supply to 5V, we couldn’t test the system under these conditions due to the risk of damaging our only microcontroller. For future iterations, testing the system with the actual power supply intended for deployment would be beneficial.

4.6 Conclusion

The detection subsystem effectively meets its primary goal of identifying the presence of honey badgers through motion detection. By integrating three distinct sensors, the system enhances detection accuracy and minimizes false triggers. The light sensor ensures that the system operates during the night when honey badgers are mostly active, conserving power by deactivating during daylight hours. The PIR sensor offers long-range detection capabilities, while the IR break beam sensor provides short-range detection. A detection event is registered when both sensors are triggered, with the information stored in code for access by other subsystems.

Chapter 5

Networking and User Interface

Prepared by: CMPSHA009

5.1 Introduction

In order for detection alerts to be remotely accessible, a server running a user interface must exist. Furthermore, this server must wirelessly communicate with the detection devices.

This section outlines the design of a user-interface and back-end communication software that aims to achieve this goal.

5.2 Requirements Analysis

5.2.1 Functional Requirements

Table 5.1 shows the functional requirements used to design the subsystem. As will be emphasized, system robustness is integral, as no data classification was considered, based on investigations on machine learning in the literature review.

FRID	Functional Requirement
FR1	User must be notified of detection event reliably
FR2	User must be notified of detection event fast
FR3	UI must be user friendly
FR4	Detection event information must be stored accessibly
FR5	User must be aware of the state of the detection devices in real-time
FR6	System must not negatively impact the environment and least strive to be energy efficient
FR7	System must minimise chance of user error

Table 5.1: List of Functional Requirements

Table 5.3 shows the functional specifications for the subsystem.

SID	Functional Specification	FRID	ATPID
S6	UI should show which devices have registered with the server and these devices should be advertised to other available devices in the peer-to-peer network.	FR5	ATP1
S7	Detection devices must periodically ping the server every five seconds. If a device has not pinged the server in a 10-second window, the UI must indicate that the device is unavailable and the device must be removed from the peer-to-peer network.	FR5	ATP2, ATP1
S8	Detection event data must be available as a labelled image, easily accessible by user.	FR3, FR4	ATP2
S9	Application must use sound to alert users of a detection event.	FR3, FR1	ATP2
S10	Detection devices must send textual alert if camera fails.	FR1	ATP4

Table 5.2: Sample Functional Specifications Table

5.2.2 Non-Functional Specifications

Table 5.2 outlines the functional specifications of the subsystem.

SID	Non-functional Specification	FRID	ATPID
S1	Server must support multiple detection devices communicating with it concurrently, using correct synchronisation principles.	FR1, FR2	AT1
S2	Application must implement scholarly user-interface design principles.	FR3	ATP6
S2	Control signals should be sent using networking technology that prioritizes speed and energy efficiency over reliable data transfer.	FR5, FR6	ATP2, ATP5
S3	Image and detection data transfer must be executed using a networking technology that prioritizes reliable data transfer over speed and energy efficiency.	FR1	ATP1, ATP5
S4	Detection devices must be able to communicate with each other as a peer-to-peer network in case one cannot reach the server.	FR1	ATP3

Table 5.3: Non-functional Specifications Mapping

5.2.3 Acceptance Test Procedures

Table 5.4 shows the acceptance test procedures and success criteria used to confirm that the subsystem met the requirements and specifications.

ATP ID	Test Procedure	Success Criteria
ATP1	Run the ESP32-cam and 2 other simulated ESP32-cams simultaneously with no detection events until each device has pinged the server four times. Thereafter, remove power from the ESP32-cam and kill the simulations. Add instrumentation to the code of the simulations and real device to output the list of peers they know about after each ping sent.	Each registration, ping and removal of a device from the peer network should be displayed by the server in real-time. Observe the simulation outputs and ESP32-cam serial monitor to confirm that all devices have the correct peer list. The entire process should not take noticeably more than 20 seconds (a ping is sent every 5 seconds).
ATP2	Run the ESP32-cam and 2 other simulated ESP32-cams simultaneously with simulated detection events for each device every 5 seconds. Wait until every device has sent four images to the server.	The images should not be corrupted. The images should be correctly labelled. A sound should be made by the UI every time a detection event occurs. The entire process should not take noticeably more than 20 seconds.
ATP3	Run a simulated device, A, that cannot connect to the server, that has a detection event every 5 seconds. Run another simulated device, B, that can connect to the previous device but cannot connect to the server. Start up the real device, C, after the other two have been running for a while. Figure 5.4 shows the situation more clearly.	The server should not receive any detection event from A until the real device is connected, illustrating that the data was routed from A, to B, to C and then to the server.
ATP4	Run a simulated ESP32-cam that suffers from a “broken camera”, that sends just the textual information of a detection to the server. Wait for three textual alerts to be sent to the server.	The server must capture the textual information and store it in some convenient format.
ATP5	System will be evaluated for its energy saving mechanisms.	System must not negatively affect the environment and must be energy efficient.
ATP6	User Interface will be evaluated for its use of scholarly design principles.	UI design must be based off of scholarly design principles.

Table 5.4: Acceptance Test Procedures

5.3 Design Choices

5.3.1 Networking

Application Type - Native Application Versus Website

The two approaches explored here are to host the user application as a website on the internet, or as a native app on the user's server.

A native application would require development of the application networking layer. A website offers the advantage of being compatible with any device, from any location with internet connectivity - however, these advantages do not matter if monitoring occurs from a fixed server. [Table 5.5](#) shows a comparison of the most important criteria for the two approaches.

Criteria	Native Application	Website Implementation	Better Approach
Protocol Flexibility and Energy Efficiency	Supports multiple transport layer protocols: UDP for fast/energy-efficient control signals and TCP for reliable data transfer.	Restricted to HTTP/HTTPS protocols, which are less flexible for real-time or low-energy communication.	Native application
System Robustness	Can be configured to launch automatically with the server, ensuring uninterrupted operation. Runs independently of external applications.	Requires manual access via browser, introducing potential delays or human error in monitoring.	Native application
Peer-to-Peer Support	Better-suited to handle peer-to-peer communications as it can manage low level networking tasks more efficiently.	Limited by browser security restrictions and bloat.	Native application
Time for Development	Since the application layer must be built, it will likely take longer to build and test.	Since the application layer already exists as HTTP/HTTPS, it will likely take less time to build. HTTP has already been thoroughly tested.	Website

Table 5.5: Comparison of Native Application vs Website Implementation

Given the system's requirements for real-time notifications, energy efficiency, peer-to-peer communication and robust operation - a native application was chosen, despite the longer development time.

Access Network - Wi-Fi versus Mobile Network

Two broad classes of access network technologies are discussed here, namely Wi-Fi and mobile cellular networks.

In the literature review it was noted that a mobile cellular network such as GSM would be ideal in terms of cost, reliability and coverage - especially in sparsely populated locations like the wilderness. Using something like GSM would require an additional GSM module to be connected to the ESP32-cam. The benefits of mobile cellular networks are not related to the fundamental architecture of the system. The same network technologies would be used in both cases. The ESP32-cam supports Wi-Fi natively and can be used easily; hence, it was decided to use Wi-Fi for this prototype, but a mobile cellular

network is recommended for a real-life deployment.

Storage of Data - Database versus OS File System

The data received by the server could be written to and queried from a database, or written to and viewed from a native directory.

Table 5.6 compares the most important categories for the two approaches.

Criteria	Database (e.g., SQL, NoSQL)	File System Storage	Better Approach
Data Complexity	Best suited for structured, relational, or large-scale data requiring queries.	Suitable for simple, infrequent or sequential data access.	File system; because most honey badgers should be repelled and not trigger the system.
Performance Overhead	Adds latency due to database features.	Minimal overhead since direct file write.	File system
Energy Efficiency	Higher energy use due to continuous processes.	Low energy impact; files are written on-demand and idle otherwise.	File system
Correct Concurrency	Comes with thoroughly tested concurrency to ensure no data races.	No inherent concurrent safety - needs to be developed and tested.	Database

Table 5.6: Comparison of Database vs File System Storage Approaches

Since it is assumed that one server is writing data, and each data item is unique, complex concurrency is not required. From the table, it is clear that the file system is justified due to its simplicity, performance, and alignment with requirements. A database would introduce unnecessary complexity without significant benefits.

5.3.2 User Interface

The UI must achieve both simplicity and functionality, so that users can quickly understand system status and respond adequately to detection events while minimizing the chance of user error. Shneiderman’s Golden Rules of Interface Design are used to ensure these requirements are met.

Layout Structure

The UI will have a three-panel layout as follows:

- **Registered devices panel:** a table-based view showing all ESP32-cam devices registered, including device ID, IP address, port status and last seen time-stamp.
- **Detection panel:** A dedicated area to display the most recent detection event with visual cues for active alerts.
- **System log panel:** A chronological record of system events for troubleshooting and monitoring.

8 This tripartite design balances several of Shneiderman’s Golden Rules, particularly informative feedback, consistency, and reduced memory load. There is a conflict between Shneiderman’s rules

of ‘reduced memory load’ and ‘universal usability’. Displaying all information simultaneously on one screen increases complexity and could overwhelm users, but requiring navigation between screens would increase memory load and reduce situational awareness. Reduced memory load is prioritized in this case, as the critical nature of detection events requires immediate visibility of all system elements.

Application of Design Principles

The UI will incorporate Shneiderman’s Golden Rules as follows:

Consistency and Standards: The interface will maintain a uniform colour scheme using neutral backgrounds for normal status and high-contrast colours for alerts. Table structures and button placement will remain consistent throughout the application, with standardized timestamp formats across all displays to ensure uniformity.

Universal Usability: The layout will prioritize simplicity and clarity over complexity, featuring high-contrast text throughout to improve readability for all users. This approach ensures the interface remains accessible to users with varying levels of technical expertise.

Informative Feedback: The system will provide visual feedback through colour shifts during alert conditions, accompanied by comprehensive logging of all events. Users will receive clear, immediate indications whenever device status changes occur.

Dialog closure: The interface will include mechanisms for alert acknowledgment and require confirmation for critical user actions, ensuring users maintain control over system operations.

User Control: Users will have manual refresh capabilities to update information on demand, along with options to dismiss alerts as needed. These features empower users to interact with the system according to their preferences.

Reduced Memory Load: The design will present all critical information simultaneously, eliminating the need for users to remember system states. User alerts will remain persistently visible until addressed, further reducing cognitive burden.

Status Monitoring

To fulfill FR5 (real-time awareness), the interface will incorporate comprehensive status monitoring features. A clear device status display will indicate active/inactive conditions for all connected devices. The system will maintain timestamp monitoring showing ‘Last Seen’ information for each device, accompanied by automatic status updates that provide immediate visual feedback whenever device status changes occur.

Alert Mechanism

To meet FR1 (reliability) and FR3 (user-friendly interface), the alert system will implement multiple notification methods. Visual indicators will trigger distinct colour changes in the detection panel during alert conditions. Users will be able to acknowledge alerts through a dedicated ‘Dismiss Alert’ button after reviewing notifications. The system will supplement visual warnings with optional audio feedback

for detection events. All alerts will remain persistently active until explicitly acknowledged by the user, ensuring no critical notifications are missed.

5.4 Final Design

5.4.1 Networking

The application layer developed for this system is encapsulated by the protocol design, message structure and code implementation. This section elaborates on these aspects.

Protocol and Message Specification

To illustrate the protocol and message structure, various scenarios are illustrated and discussed. The message format is JSON in the code, and is visually represented as JSON messages in this section. Much of the behaviour of the server and detection devices will be demonstrated in this section, with lower level details discussed in the 'Code Implementation' section.

Camera errors are common in the ESP32-cam. Wireless communication errors are common in general. This is why there are scenarios involving camera and network errors. The main goal of this system is to be robust in getting detection events to the user - necessitating planning for camera and wireless communication errors.

Register With Server Figure 5.1 shows the sequence diagram for a successful registration and the message structures for the relevant messages. In this scenario, it is assumed that two prior detection devices registered with the server, and now device 'ESP32-CAM-0068' intends to register.

Each detection device registers with the server using a unique user ID, IP address and port number. This information is used by the server to inform peers about how to contact each other, and to send control signals to each detection device. The server uses the value of the 'message_type' JSON field to understand what type of message it is, and do the required action.

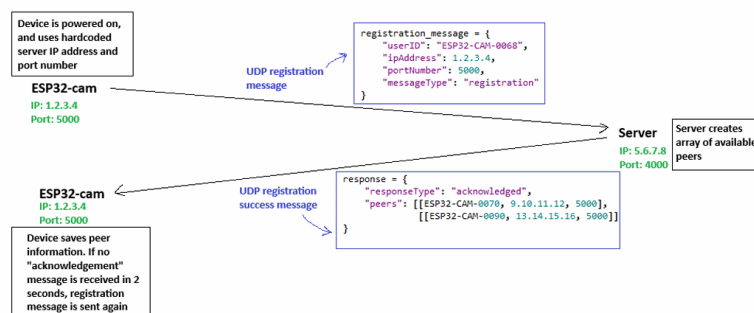


Figure 5.1: Sequence Diagram for Registration with Server

Ping Server In order to have knowledge of device liveness, the detection devices 'ping' the server every five seconds. The server responds with a list of the updated peer list since the last ping. If the server does not receive a ping from a specific device in a ten-second window, the server removes it from its peers list - the device is unavailable.

Continuing with the previous scenario, we assume that one new peer registers with the server. Then peer 'ESP32-CAM-0068' pings the server, and receives an acknowledgement with the new peer list, now having three peers instead of two. This is shown by Figure 5.2

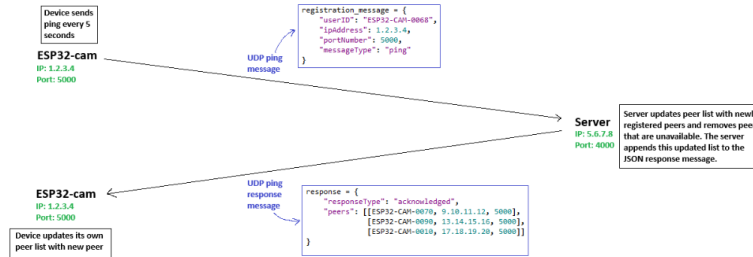


Figure 5.2: Sequence Diagram for Pinging Server

Detection Event with successful TCP connection (camera working and not working)

Upon a detection event, the given ESP32-cam must successfully establish a TCP connection with the server in order to transfer information to it reliably. Carrying on from the last scenario, we assume that a detection event occurs in 'ESP32-CAM-0068'. We assume that the TCP connection is successful. The device sends a JSON message over the TCP message with details of the detection event, and then transfers the image over the same TCP connection. Figure 5.3 shows this process.

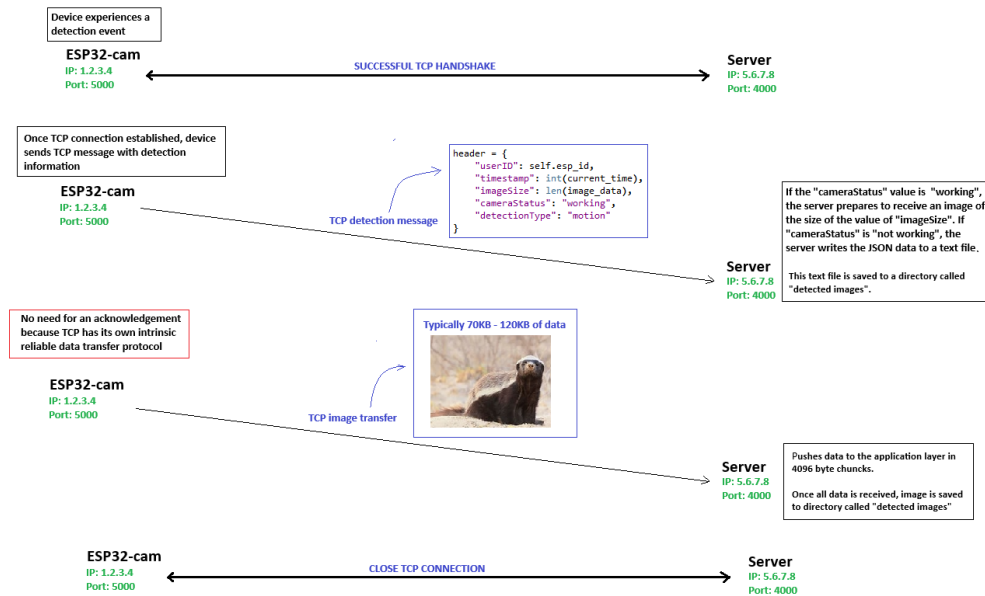


Figure 5.3: Sequence Diagram for Detection Event

Received data is saved to a directory called 'detected images' for later review. If the 'cameraStatus' value in the TCP detection message is 'not working', the server saves the JSON message data in a text file to the 'detected images'.

TCP is far more robust than UDP, using timeouts, acknowledgements, congestion control and flow control. This also means that it is more energy intensive, slower and may be throttled by a severely congested network. This is why TCP is only used in this case, when data integrity is essential.

Detection Event with unsuccessful TCP connection (peer-to-peer case) Each ESP32-cam acts as a peer and a server. Hence, the TCP file transfer discussed in the previous scenario works the same way between peers.

Figure 5.4 shows two network scenarios to illustrate what occurs when a detection device A fails to send a detection message and image to the server. The peer-to-peer network is used to try to get the data to the server through one or more other peers. This is accomplished as follows:

- Device A attempts to establish a TCP connection with the server, and fails.
- Device A iteratively tries to start TCP connections with the peers it knows about until one is successful. In the first case, A successfully connects to B on its first try. In the second case, A fails to connect to B, and then tries to connect to C and succeeds.
- In the first case, B connects to the server and the data is delivered successfully. In the second case, C cannot establish a connection to the server. C starts the process over again, iteratively trying to connect to its peers (except A).
- C successfully connects to B, and sends the detection to B, which is then sent from B to the server.

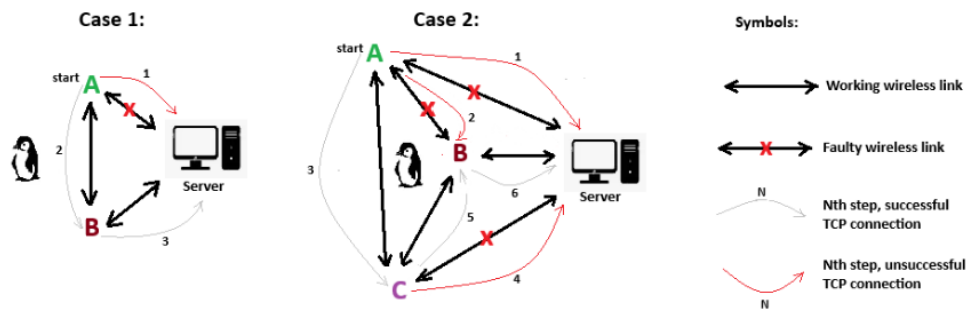


Figure 5.4: Peer-to-peer Network Mechanism

This method works, as long as there is a route from the starting device to the server in the network. In a network of more detection devices, the likelihood of a successful route being possible increases. Due to the immense scale that any particular colony of penguins could occupy, it is likely that many detection devices would be employed, and form a robust peer-to-peer network.

Code Implementation

The following programs will be discussed in this section:

- Server program ('SERVER_GUI.py'), written in python. The GUI aspects will only be discussed in the UI section.
- ESP32-cam program ('ESP_CODE_PP.ino'), written in C++.
- Several python programs that simulate ESP32-cam devices with different errors (camera not working for example).

Simulated detection devices had to be created to create a network with multiple devices and to simulate network and camera errors.

Python was chosen to create the server and simulated ESP32-cam programs because of ease of use.

Server The server is implemented as a python class with a function for handling UDP messages and a function for handling TCP messages. These two methods run indefinitely, and are started separately as threads. The class has a field called “devices”, an array that stores arrays of peer information, namely the user ID, peer IP address, and peer port number. It also has a field called “devices_lock”, which is the lock that must be acquired to write to the devices array, as there are more than one function that may write to the devices array.

Since UDP is so fast, the UDP method simply poles its UDP socket. When it gets a UDP message, it reacts and goes back to listening for UDP messages. The UDP socket is opened on the IP address of the server, and the port 1234. These two values are hardcoded into all ESP32-cams and ESP32-cam simulations. Each UDP message will have a header field called “message_type” as seen in the previous section. If the value of this field is “registration” or “ping”, the UDP thread sends a response (as seen in the previous section) through the same UDP socket and adds the requesting device’s info to the devices array. The UDP socket timeout is set to one second. This means that after one second of listening, the code jumps into a socket timeout exception block. Within this block, a function called “cleanup_inactive()” is called. This function acquires the devices array lock and then deletes peers from a public peer array that have not pinged the server in a ten-second window.

A TCP socket is bound to the IP address to the server, and the port number 1235, which is again known by all peers. The TCP thread uses a more robust method for handling incoming messages, since it is slower, and concurrent access problems can hence arise. [Figure 5.5](#) shows the way it works.

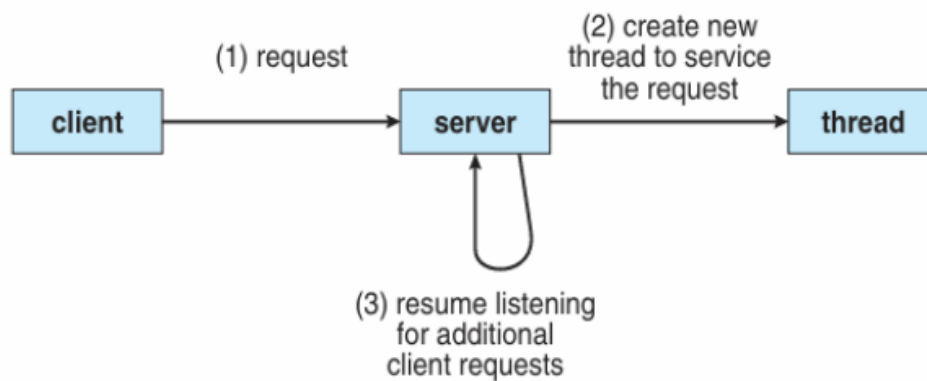


Figure 5.5: Server Architecture

This is realised by starting a new thread every time a TCP message is received that deals with the message, while the original thread keeps listening for new TCP messages.

When the TCP socket receives a JSON message, it checks the ‘cameraStatus’ header value. If it is ‘broken’, the JSON data is used to create a text file containing the detection information. An example of this is shown in [Figure 5.6](#)

```

Camera malfunction detected|
Device: ESP32-CAM-0012
Time: 2025-05-22 14:03:37
Detection Type: motion

```

Figure 5.6: Text File Log for Broken Camera Detection Event

This file is saved to a directory called ‘detected_images’, in the same directory as the server program. If ‘cameraStatus’ is not ‘broken’, the thread waits for the detection image to be sent. This image is received in 4096 byte chunks, and saved as an image to the ‘detected_images’ directory. The name of the image is set to valuable detection information as shown in [Figure 5.7](#)



Figure 5.7: Labelled Image from Detection

ESP32-cam and Simulated ESP32-cam [Figure 5.8](#) shows the general behaviour for the ESP32-cam and the simulated ESP32-cams. The real ESP32-cam does not actually use threads or interrupts, but just executes each task sequentially in an infinite loop, which is a good approximation for [Figure 5.8](#) under low load conditions.

The simulated ESP32-cams use multithreading similarly to what was described in the previous section.

Four simulation programs were created for testing and proof of concept purposes:

- ‘ESP_SIMULATION_SUCCESS.py’, which simulates a device that experiences no failed TCP connections to the server and no failed camera captures.
- ‘ESP32_SIMULATION_TCP_FAILURE.py’, which simulates a device that always fails to establish a TCP connection with the server, and hence must forward detection data to a peer.
- ‘ESP_SIMULATION_BROKEN_CAMERA.py’, which simulates a device that always fails to take pictures, resulting in the device sending a JSON message with no image. In the real device, the camera function returns 0 if it fails, which can be checked.
- ‘ESP_SIMULATION_MIDDLE.py’, which simulates a device that experiences no detection events, and is an ideal for just forwarding messages within the peer-to-peer network.

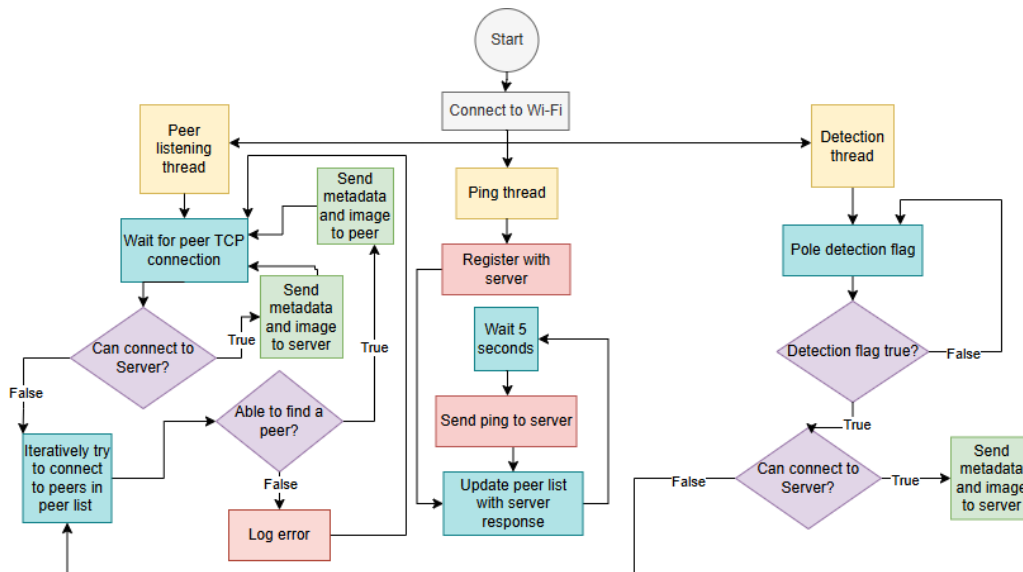


Figure 5.8: Flow Chart for General Behaviour of ESP32-cam Device Behaviour

All programs in this section output useful information to the serial monitor (for the real ESP32-cam) or terminal (for the server and simulated ESP32-cams). All code can be found using at [Github Link](#)

5.4.2 User Interface

Functional Overview

Figure 5.9 shows the start-up appearance of the user-interface. The system log displays what would be the terminal output of the server.

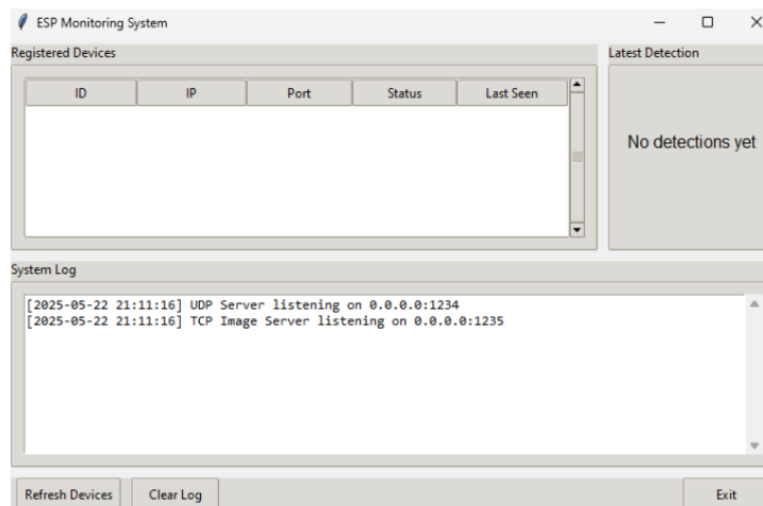


Figure 5.9: User Interface Start Up

Figure 5.10 shows the system log frame for when a device registers, updates the server, sends detection information and exits the network.

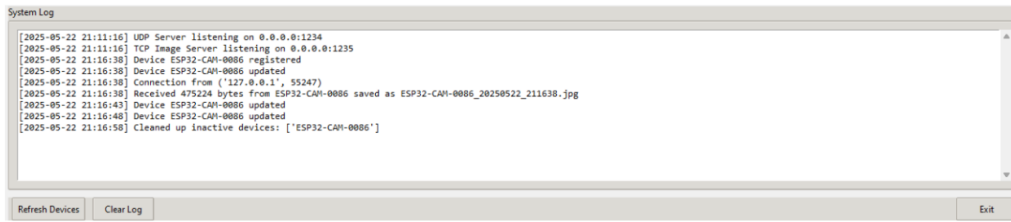


Figure 5.10: User Interface System Log

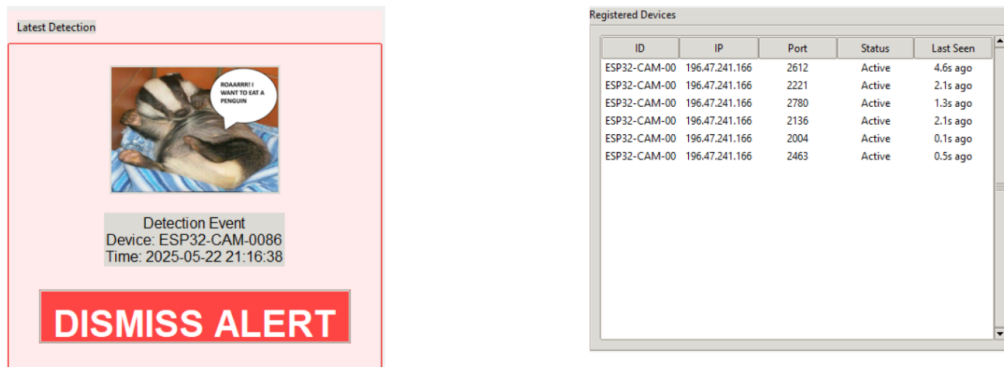


Figure 5.11: Detection Frame During Alert and Registered Devices Frame

Figure 5.11 shows the detection frame when a detection event is received. In this case, a dummy image was used for illustration purposes. Figure 5.11 also shows the registered devices frame when a few devices are part of the network.

The 'Clear Log' button removes all text from the system log frame. The 'Refresh Devices' button updates the last seen values in the registered devices frame. The 'Dismiss Alert' button allows the user to remove the alert from the latest detection frame once they have dealt with the detection.

Code Implementation

The GUI was implemented using the Python 'tkinter' library. The 'SERVER_GUI.py' file is the server code wrapped up in the GUI logic. The GUI has a queue that must be correctly managed, or the program will enter a deadlock.

5.5 Results and Testing

Table 5.7 shows the results from acceptance testing.

ATP ID	Test Procedure	Success Criteria	Result
ATP1	Run the ESP32-cam and 2 other simulated ESP32-cams simultaneously with no detection events until each device has pinged the server four times. Thereafter, remove power from the ESP32-cam and kill the simulations. Add instrumentation to the code of the simulations and real device to output the list of peers they know about after each ping sent.	Each registration, ping and removal of a device from the peer network should be displayed by the server in real-time. Observe the simulation outputs and ESP32-cam serial monitor to confirm that all devices have the correct peer list. The entire process should not take noticeably more than 20 seconds (a ping is sent every 5 seconds, and we are waiting for 4 pings).	Passed. Figure 5.12 shows the system log of four updates from the devices, which took 20 seconds. Each registration showed, and each device was removed from the network shortly after being ‘powered off.’ Figure 5.12 shows the real device printing two of its peers out to the serial monitor after each ping. The peer list accurately showed how many peers were in the network.
ATP2	Run the ESP32-cam and 2 other simulated ESP32-cams simultaneously with simulated detection events for each device every 5 seconds. Wait until every device has sent four images to the server.	The images should not be corrupted. The images should be correctly labelled. A sound should be made by the UI every time a detection event occurs. The entire process should not take noticeably more than 20 seconds.	Passed. The entire process took 20 seconds. Images taken by the real device, and dummy images sent by the simulated devices were labelled and stored correctly. A sound was made upon every detection event.
ATP3	Run a simulated device, A, that cannot connect to the server, that has a detection event every 5 seconds. Run another simulated device, B, that can connect to the previous device but cannot connect to the server. Start up the real device, C, after the other two have been running for a while. Figure 5.4 shows this situation more clearly.	The server should not receive any detection event from A until the real device is connected, illustrating that the data was routed from A, to B, to C and then to the server. Code instrumentation must verify that this route was taken.	Passed. Figure 5.13 shows the terminal, serial and UI outputs that verify that this test was passed.
ATP4	Run a simulated ESP32-cam that suffers from a ‘broken camera’, that sends just the textual information of a detection to the server. Wait for three textual alerts to be sent to the server.	The server must capture the textual information and store it in some convenient format.	Passed. The UI showed an alert for the detection and saved the metadata to a text file in the ‘detected_images’ directory.
ATP5	System will be evaluated for its energy saving mechanisms.	System must not negatively affect the environment and must be energy efficient.	Passed. TCP and UDP were used to minimise energy use.
ATP6	System will be evaluated for its use of scholarly user interface principles.	Scholarly user interface principles must have been used.	Passed. Nielsen and Scheiderman’s UI design principles were implemented.

Table 5.7: Acceptance Test Procedures with Results

```

[2025-05-23 18:57:20] Device ESP32-CAM-001 updated
[2025-05-23 18:57:24] Device ESP32-CAM-0091 updated
[2025-05-23 18:57:24] Device ESP32-CAM-0059 updated
[2025-05-23 18:57:25] Device ESP32-CAM-001 updated
[2025-05-23 18:57:29] Device ESP32-CAM-0091 updated
[2025-05-23 18:57:29] Device ESP32-CAM-0059 updated
[2025-05-23 18:57:30] Device ESP32-CAM-001 updated
[2025-05-23 18:57:34] Device ESP32-CAM-0091 updated
[2025-05-23 18:57:34] Device ESP32-CAM-0059 updated
[2025-05-23 18:57:35] Device ESP32-CAM-001 updated
[2025-05-23 18:57:39] Device ESP32-CAM-0091 updated
[2025-05-23 18:57:39] Device ESP32-CAM-0059 updated

Sent ping: {"userID":"ESP32-CAM-001","ip":
Current Peer List:
-----
1: ESP32-CAM-0091 at 196.47.241.166:3164
2: ESP32-CAM-0059 at 196.47.241.166:3085
-----
Sent ping: {"userID":"ESP32-CAM-001","ip":
Current Peer List:
-----
1: ESP32-CAM-0091 at 196.47.241.166:3164
2: ESP32-CAM-0059 at 196.47.241.166:3085
-----
Sent ping: {"userID":"ESP32-CAM-001","ip":

```

Figure 5.12: System Log Output for ATP1 Showing Device Registration and Pings and ESP32-cam Serial Monitor Output Showing Peer List Updates

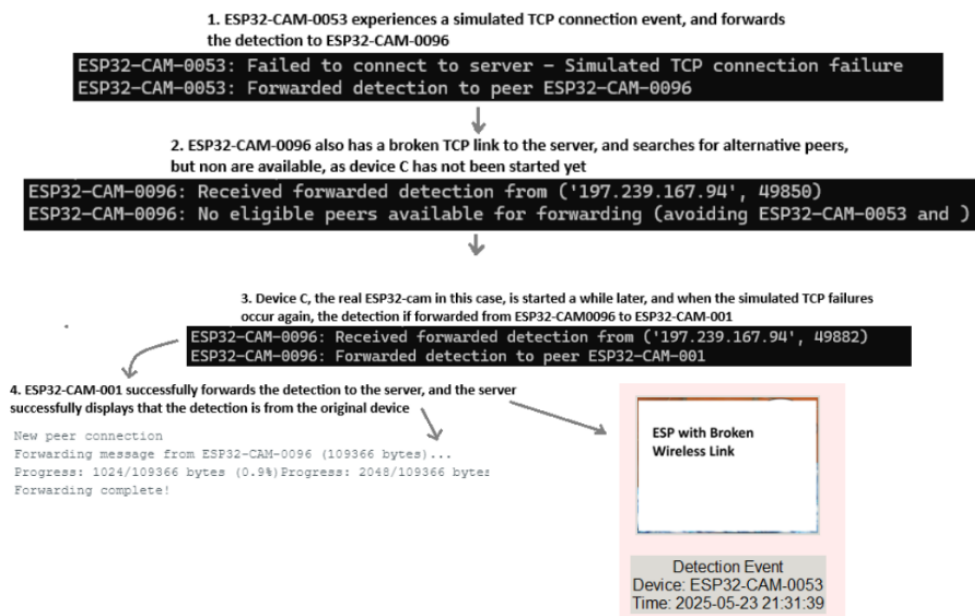


Figure 5.13: Terminal, Serial and UI Outputs Verifying Peer-to-Peer Routing (ATP3)

5.6 Conclusion

From analyzing the literature, it was clear that incorporating IoT into the problem of deterring honey badges is essential. It is also clear, that classification must include something like machine learning. In this instance, it was decided that machine learning would not be used, but that IoT should still be utilized.

Without classification, it falls on the user to classify the animal. This leaves this subsystem with one main objective: to get detection information to the user robustly and quickly.

To achieve this goal, while leveraging IoT, a peer-to-peer network was designed and implemented. This peer-to-peer network adds robustness to the system. In future research, more features of the per-to-peer network could be implemented, such as activating all nearby ESP32-cam repellent systems upon a detection event, to create a potentially more effective, and distributed repellent mechanism. Many existing internet ideas could also make the peer-to-peer network faster, such as computing shortest paths in the network.

The subsystem passed all ATPs, successfully demonstrating that a design such as this one is feasible for this problem.

Chapter 6

System Interface

Integrating the detection subsystem with the networking and user interface components proved challenging, primarily because it required consolidating all functionalities onto a single microcontroller. While the detection code independently stored motion detection variables successfully, merging it with the networking and UI code caused the ESP32-CAM to fail in establishing connections and delivering the expected outputs. As a result, the subsystems had to operate separately.

Figure 6.1 below, indicates the output from attempts to run the combined code for the detection and networking subsystems.

```
entry 0x400805f0
System ready - Initializing components as needed
BH1750 initialized
Light: 0.00 lx
System ON - Dark conditions
Attempting WiFi connection...
Connecting to WiFi..Guru Meditation Error: Core  1 panic'ed (LoadProhibited). Exception was unhandled.

Core  1 register dump:
PC      : 0x400925bb  PS      : 0x00060030  A0      : 0x800928e4  A1      : 0x3ffd0940
A2      : 0xffffffff  A3      : 0xffffffff  A4      : 0x3ffdc884  A5      : 0x3ffd178c
A6      : 0x00000000  A7      : 0x00000001  A8      : 0xffffffff  A9      : 0x3ffd0a40
A10     : 0x3ffdc884  A11     : 0x000013a2  A12     : 0x00000400  A13     : 0x3ffdbe0c
A14     : 0x00000000  A15     : 0x00000040  SAR     : 0x0000001a  EXCCAUSE: 0x0000001c
EXCVADDR: 0x0000000f  LBEG    : 0x4008b638  LEND    : 0x4008b654  LCOUNT   : 0x00000000

Backtrace: 0x400925b8:0x3ffd0940 0x400928e1:0x3ffd0960 0x4009357b:0x3ffd0980 0x400936ec:0x3ffd09a0 0x40084238:0x3ffd09c0 0x400
```

Figure 6.1: ESP32-CAM system intrface output

As it can be seen from the output code above, the sensor subsystem initializes well with light readings from the light sensor being displayed on the output; however, the microcontroller fails to connect to the wifi, producing an error. This could be due to the microcontroller's limited memory capacity, as it would have to interface with all of the sensors, capture an image, and connect to wifi for data transmission.

Chapter 7

Conclusions

To help sustain new African Penguin colonies, a system was designed to repel honey badgers from these colonies. The honey badger is extremely courageous, making this problem especially difficult.

This report began by outlining the problem and its scope and limitations. Most critically, the time frame for this project was very short. Not having access to the real environments also meant that the effectiveness of repellents is unknown. This is emphasized by the literature review, which repeatedly demonstrates that repellent techniques are speculatively effective across different animal species.

The literature review elucidated that honey badgers are nocturnal, sensitive to light and to certain smells. This motivated the repellent system being a bright light and diffusion of a known irritating smell. The literature also showed that classifying animals using machine learning probably requires hardware and time that is above the limitations of this report. This inspired the idea to try to leverage IoT in a different way, that relies on robust detection, networking and a user-interface that minimizes human error.

Chapter 4 showed that robust detection within the limitations of this report is possible. The section outlines a subsystem that not only detects motion, but whether or not the honey badger was not repelled. This is critical, as it acknowledges the reality that the repellent system may not work.

This acknowledgment is continued in Chapter 5. Provided the repellent does not work, the user must be notified about it, classify it, and react. This is the last resort, and something that would require somebody watching the system continuously. This chapter elaborates on how networking and user-interface design principles can make the system robust and fast.

The subsystems in Chapters 4 and Chapter 5 ended up not integrating, as it seemed that an oversight was made on the ability of one micro-controller to handle both subsystems. Both subsystems, however, were successful in demonstrating their own goals. Unfortunately, the repellent subsystem could not be developed, but it's conception was the product of the entire team, and is consistent with the read literature.

Chapter 8

BOM, GA and Github Link

8.1 Bill of materials

Table 8.1: Bill of Materials

Component	Price (R)	Quantity	Total (R)
ESP32-CAM Development Board	182.00	1	182.00
ESP32-CAM-MB Micro USB Programmer	35.00	1	35.00
HC-SR501 PIR Sensor	27.00	2	54.00
IR Break Beam Sensor Module	18.00	2	36.00
BH1750FVI Light Sensor	40.00	2	80.00
4.7 k Ω Resistor	0.69	2	1.38
1 k Ω Resistor	0.69	2	1.38
100 Ω Resistor	0.70	1	0.70
Red LED	1.09	1	1.09
2N2222A BJT Transistor	3.00	1	3.00
Breadboard Jumpers	20.00	1	20.00
DC-DC 5A Adjustable Buck Regulator	38.00	2	76.00
Micro Submersible Water Pump DC 3V–5V	34.81	2	69.62
Veroboard	80.00	1	80.00
Total			640.17

8.2 GAS

Table 8.2: Animal Detection Subsystem – Graduate Attributes Mapping

Animal Detection Subsystem – NGSLES001	
Graduate Attribute	Evidence in Report
GA3: Engineering Design	Chapter 4, Section 4.3 extensively provides the engineering design work.
GA7: Sustainability and Impact of Engineering Activity	Chapter 4, Sections 4.1, 4.3.1 cover the incorporation of a light sensor to be used to power the system only at night, to save power and increase electronic components' lifespan.
GA8: Individual, Team, and Multidisciplinary Working	Participation in team meetings and discussions (refer to Teams channel) and consistent attendance in consultations
GA10: Engineering Professionalism	Completion of all submission activities, including final report and presentation

Graduate Attribute	Sections Met in Networking and UI (CMPSHA009)
GA3	Sections 5.3 and 5.4.
GA7	D-School activities, optimising energy efficiency of network where possible and evaluating network for environmental harm. Section 5.4.1 explains how UDP and TCP were used in a way that optimises for energy efficiency.
GA8	Engagement in Teams channel and regular meetings with Team. The two subsystems were ran on the same microcontroller, which required integration discussions.
GA10	Submissions were submitted on time and there was professional communication between group members, course staff and stakeholders.

Table 8.3: Graduate Attributes Met in Networking and UI (CMPSHA009)

8.3 Github Link

[Github Link](#)

Bibliography

- [1] R. B. Sherley, A. B. Makhado, R. J. Crawford, C. Hagen, J. Kemper, K. Ludynia, and L. J. Waller, “The african penguin *spheniscus demersus* should be considered critically endangered,” *Ostrich*, vol. 95, no. 3, pp. 181–187, 2024.
- [2] P. A. Whittington, J. H. Hofmeyr, and J. Cooper, “Establishment, growth and conservation of a mainland colony of jackass penguins *spheniscus demersus* at stony point, betty’s bay, south africa,” *Ostrich*, vol. 67, pp. 144–150, 1996.
- [3] R. K. Brooke, *South African Red Data Book – Birds*, ser. South African National Scientific Programmes Report No. 97. Pretoria, South Africa: Council for Scientific and Industrial Research, 1984.
- [4] R. Crawford, R. Altwegg, B. J. Barham, P. J. Barham, J. M. Durant, B. M. Dyer, and et al., “Collapse of south africa’s penguins in the early 21st century,” *African Journal of Marine Science*, vol. 33, no. 1, pp. 139–156, 2011.
- [5] R. J. M. Crawford, A. B. Makhado, and W. H. Oosthuizen, “Bottom-up and top-down control of the benguela ecosystem’s seabirds,” *Journal of Marine Systems*, 2018.
- [6] M. . Guardian, “Killing spree: Honey badger wipes out 11 endangered penguins at cape reserve,” 2024, [Online]. Available: <https://mg.co.za/the-green-guardian/2024-04-18-killing-spreed-honey-badger-wipes-out-11-endangered-penguins-at-cape-reserve/>.
- [7] C. Begg, K. Begg, and J. Kingdon, *Subfamily Mellivorinae. Ratel*, J. Kingdon, D. Happold, T. Butynski, M. Homann, M. Happold, and J. Kalina, Eds. London, UK: Bloomsbury Natural History, 2013, vol. V.
- [8] J. M. Gil-Sánchez, F. J. Herrera-Sánchez, J. Rodríguez-Siles, J. M. Sáez, M. Díaz-Portero, Arredondo, B. Álvarez, I. Cancio, J. de Lucas, S. Castillo, and E. McCain, “Feeding specialization of honey badgers in the sahara desert: A trial of life in a hard environment,” *Diversity*, vol. 12, no. 2, 2020.
- [9] M. Live, “Honey badger,” 2025, [Online]. Available: https://www.mpalalive.org/field_guide/view/honey_badger.
- [10] H. Kruuk, “Notes on food and foraging of the honey badger *mellivora capensis* in the kalahari gemsbok national park,” *Koedoe*, vol. 26, pp. 153–157, 1983.
- [11] A. Johnson, “The effects of tactile and visual deterrents on honey badger predation of beehives,” Master’s thesis, City University of New York, 2019.

- [12] E. Team, “What smells don’t badgers like?” 2025, <https://enviroliteracy.org/what-smells-dont-badgers-like/>.
- [13] M. E. Smith, J. D. C. Linnell, J. Odden, and J. E. Swenson, “Review of methods to reduce livestock depredation ii. aversive conditioning, deterrents and repellents,” *Acta Agriculturae Scandinavica, Section A — Animal Science*, vol. 50, no. 4, pp. 304–315, 2000.
- [14] M. Bomford and P. H. O’Brien, “Sonic deterrents in animal damage control: a review of device test and effectiveness,” *Wildlife Society Bulletin*, vol. 18, pp. 411–422, 1990.
- [15] T. W. Rock, “An evaluation of seasonal coyote control techniques and sheep losses in saskatchewan,” MSc Thesis, University of Nevada, Reno, NV, 1978.
- [16] S. B. Linhart, R. T. Sterner, G. J. Dasch, and J. W. Theade, “Efficacy of light and sound stimuli for reducing coyote predation upon pastured sheep,” *Protection Ecology*, vol. 6, pp. 75–84, 1984.
- [17] J. M. N. LeFranc, M. B. Moss, K. A. Patnode, and I. W. C. Sugg, *Grizzly bear compendium*. Missoula, MT: The Interagency Grizzly Bear Committee, 1987.
- [18] C. R. Gustavson and et al., “Coyote predation control by aversive conditioning,” *Science*, vol. 184, pp. 581–583, 1974.
- [19] P. N. Lehner and et al., “Tests for olfactory repellents for coyotes and dogs,” *The Journal of Wildlife Management*, vol. 40, no. 1, pp. 145–150, 1976.
- [20] P. G. Hatfield and J. W. Walker, “An evaluation of pred-x eartag in protection of lambs from coyote predation,” *Sheep Research Progress Report*, no. 3, p. 193, 1994.
- [21] J. L. Dalle-Molle and J. C. V. Horn, “Bear–people conflict management in denali national park, alaska,” in *Bear–People Conflicts: Proceedings of a Symposium on Management Strategies*, M. Bromley, Ed. Yellowknife: Dept of Renew. Res., 1989, pp. 129–132.
- [22] R. Sumathi, P. Raveena, P. Rakshana, P. Nigila, and P. Mahalakshmi, “Real time protection of farmlands from animal intrusion,” in *2022 IEEE World Conference on Applied Intelligence and Computing (AIC)*, Sonbhadra, India, 2022, pp. 859–863.
- [23] R. R. Kommasani, K. B. Malla, S. P. Vasam, T. Bhukya, and A. M. Karlapudy, “Solar based fencing and animal detection system for agricultural fields,” *International Journal of Emerging Technologies and Innovative Research*, vol. 6, no. 2, pp. 183–190, 2019, [Online]. Available: <https://www.jetir.org/view?paper=JETIR1902B25>.
- [24] S. H. Fritts, W. J. Paul, L. D. Mech, and D. P. Scott, *Trends and management of wolf–livestock conflicts in Minnesota*, ser. Research Publication No. 181. Minneapolis, MN: US Fish and Wildlife Service, 1992.
- [25] C. M. Gillin, F. M. Hammond, and C. M. Peterson, “Evaluation of an aversive conditioning technique used on female grizzly bears in the yellowstone ecosystem,” in *International Conference on Bear Research and Management*, vol. 9, 1994, pp. 503–512.

- [26] A. Mishra and K. K. Yadav, "Smart animal repelling device: Utilizing iot and ai for effective anti-adaptive harmful animal deterrence," in *BIO Web of Conferences*, vol. 82. EDP Sciences, 2024.
- [27] D. Adami, M. O. Ojo, and S. Giordano, "A monitoring application for animal repelling devices in smart agriculture," *IEEE Access*, vol. 9, pp. 132 125–132 139, 2021.
- [28] U. Dampage, R. Thajudeen, S. Jasenthuliyana, and J. Jayawardena, "Environmental monitoring and assessment," *Environmental Monitoring and Assessment*, vol. 193, pp. 1–14, 2021.
- [29] K. Balakrishna, F. Mohammed, C. R. Ullas, C. M. Hema, and S. K. Sonakshi, "Global transitions proceedings," *Global Transitions Proceedings*, vol. 2, no. 2, pp. 169–174, 2021.
- [30] A. J. Simla, R. Chakravarthi, and L. M. Leo, "Agricultural intrusion detection (aid) based on the internet of things and deep learning with the enhanced lightweight m2m protocol," *Soft Computing*, pp. 1–12, 2023.
- [31] G. Moallem, D. D. Pathirage, J. Reznick, J. Gallagher, and H. Sari-Sarraf, "Knowledge-based systems," *Knowledge-Based Systems*, vol. 216, 2021.
- [32] S. Thangavel and C. S. Shokkalingam, "Journal of ambient intelligence and humanized computing," *Journal of Ambient Intelligence and Humanized Computing*, vol. 13, no. 6, pp. 3065–3081, 2022.
- [33] A. V, H. R, R. C. P, Y. P. Y, H. P. Menon, and S. S. P, "Virtual fencing to prevent human wild conflict," in *2023 International Conference on Innovations in Engineering and Technology (ICIET)*, 2023, pp. 1–6.
- [34] M. Gor, J. Vora, S. Tanwar, S. Tyagi, S. Kumar, and M. S. R. Afonso, "Wireless iot-based animal tracking and management system for farms," in *International Conference on Computing and Network Communications (CoCoNet 2017)*, 2017.
- [35] Thethingsnetwork.org, "The things network," 2019, [https://www.thethingsnetwork.org/International Conference on Communication and Electronics Systems \(ICCES\)](https://www.thethingsnetwork.org/International%20Conference%20on%20Communication%20and%20Electronics%20Systems%20(ICCES)), Coimbatore, India, 2023, pp. 1635-1639, doi: 10.1109/ICCES57224.2023.10192812.
- [36] U. Divya and Praveen.M, "Iot-based wild animal intrusion detection system," *Journal on Recent and Innovation Trends in Computing and Communication*, vol. 6, no. 7.
- [37] H. R. D. M. M. G. K. R. S. Bhatta, S. Goud Bathini and D. Ajitha, "Prototype for smart crop protection against wild animals," *2023 8th International Conference on Communication and Electronics Systems (ICCES), Coimbatore, India, 2023*, pp. 1635-1639, doi: 10.1109/ICCES57224.2023.10192812.